UNIVERSITY OF BRISTOL

School of Mathematics

Algorithms and Programming in C(++) and R
MATH10017
(Paper code MATH-10017W)

Winter 2025 1 hour 30 minutes

This paper contains two sections: Section A and Section B.
Each section should be answered in a separate booklet.

Section A contains FIVE questions and Section B contains TWO questions
All SEVEN answers will be used for assessment.

Calculators are not permitted.

Do not turn over until instructed.

Section A: Short Questions

A1. (**8 marks**)

(a) (**2 marks**) In C programming language, which one of the following statements is **wrong**?

(A) Elements in an array are stored in a contiguous section of memory.

(B) The name of an array can be used as a pointer to its first element.

(C) The index of the first element of an array is 1.

(D) Accessing memory outside the bounds of an array may cause undefined behaviour.

(E) C code must be compiled, not interpreted.

(b) (**3 marks**) In R, what is the main difference between a `vector` and a `list`? Provide a short example to illustrate.

(c) (**3 marks**) In R, what is the index of the first element of a vector? Give a short example to illustrate.

A2. (**8 marks**) Read the following C program and answer questions.

```
#include <stdio.h>

void main(){
    int f(int a, int x){
        printf("function f\n"); // L1
        return a + 2*x;
    }
    int a = 2, b = 1;
    int ff = f(a, f(b, a)); // *
    printf("%d\n", ff); // *

}
```

(a) (**3 marks**) What will happen if you try to compile this code? Explain the issue, and propose a fix. You are not allowed to change the lines marked by *. Your fix should also not alter the intended outcome of the code.

(b) (**2 marks**) After the fix, what is the output of the program?

(c) (**3 marks**) Draw the stack memory layout when the program running the like marked with the comment "L1" **the second time**. Your diagram must include all local variables and their values.

A3. (**8 marks**)

The following C code sums overs all rows of a 3 by 3 matrix. The matrix is stored in `A` in row major order. It prints out a column vector, for which the i-th element is the sum of the i-th row.

```
int elem(int M[], int i, int j){
    __YOUR CODE HERE__
}
void rowsum(int A[]){
    int sum[3] = {0, 0, 0};
    for (int i = 0; i < 3; i++){
        for (__YOUR CODE HERE__){
            sum[i] += __YOUR CODE HERE__;
        }
    }

    //print the result
    for (int i = 0; i < 3; i++){
        printf("%d \n", sum[i]);
    }
}

void main(){
    // A stores a 3 by 3 matrix in row-major order.
    int A[] = {1,2,3, 2,1,3, 3,3,1};

    rowsum(A);
}
```

(a) (**2 marks**) Suppose that the `elem` function returns the i, j-th element of a 3 by 3 matrix `M` stored in row-major order. Complete `elem` by using **one line of code**. Use zero-based indexing.

(b) (**3 marks**) Complete the missing lines in the `rowsum` function.

(c) (**3 marks**) Suppose I use the above algorithm to compute the row sum of an m by n matrix. It takes t seconds to compute the outcome. Approximately, how much time would the algorithm take when computing row sum on a $2m$ by $2n$ matrix? Briefly justify your answer.

A4. (**8 marks**) Consider an R function called `nearest_index` which takes as inputs a scalar number `x` and a numeric vector `v`. The function should return the index `k` of the element in `v` that is **closest** to `x`. For example:

```
> z <- c(3, -10, 7)
> nearest_index(x = -12, v = z) # closest to -12 is -10 (index 2)
[1] 2
> nearest_index(x = -1, v = z)  # closest to -1 is 3 (index 1)
[1] 1
```

Read the following R code and answer the questions.

```
nearest_index <- function(x, v){
  ng <- length(v)
  best_k <- 1
  best_d <- Inf # + Infinity

  for(ii in 1:ng){
    d <- abs(x - v[ii])
    if (d < best_d) {
    # YOUR CODE HERE
    }
  }

  return(best_k)
}
```

(a) **(3 marks)** Complete the function so that it works as described above.

(b) **(3 mark)** Explain what will be the output of this code:

```
x <- c(2, 1)
for(ii in x){
 print( nearest_index(x = ii, v = c(-1, 3, 0)) )
}
```

(c) **(2 marks)** Assume that we know that the elements of `v` are increasing (i.e., `v[1] < v[2] < ...`). Make your function more efficient by exploiting this fact (it should be sufficient to add one line of code).

A5. (**8 marks**) The Frobenius norm of a matrix $A \in \mathbb{R}^{m \times n}$ is

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij}^2}.$$

Answer the following questions in R.

(a) (**4 marks**) Write an R function `frobenius_norm(A)` that computes the Frobenius norm of a numeric matrix A using at least one `for()` loop and without calling `norm()`.

(b) (**2 marks**) Write a vectorised version of `frobenius_norm(A)` that avoids explicit loops or calls to `sapply`, `lapply`, etc.

(c) (**2 marks**) Explain what, in your opinion, will be the output of this code:

```
X <- matrix(c(1, 2, 2, 1), ncol = 2, nrow = 2, byrow = TRUE)

z <- frobenius_norm(X - t(X))

z:10
```

Section B: Longer Questions

Please use a new booklet for this Part.

B1. (**30 marks**) Consider a meteorological dataset that contains the record of rainfall of m cities on n dates. The dataset is stored as a matrix $A \in \{0,1\}^{m \times n}$. $A_{i,j} = 1$ indicates that it rained in the city i on the j-th day. $A_{i,j} = 0$ means the city i did not rain on the j-th day.

- Suppose $m = 100$, $n = 365$. Use zero-based indexing in this question.
- You are given the skeleton C code below.
- Function `elem` has been provided to you. `elem(M, i, j)` returns $M_{i,j}$.

```
int elem(int M[], int i, int j){
  // function body omitted
}

// YOUR CODE GOES HERE

void main(){
    // A is a 100 by 365 matrix stored in row-major order.
    int A[] = {1,0,0,1,0,1,1,0, /*the rest of A is omitted*/};

    int i = 23, j = 17;
    // predict the rain fall of city i on the j-th date
    int rain = predict(A, i, j);
    printf("%d\n", rain);
    // ... other function calls
}
```

(a) (**10 marks**) Write a function `int raininess(int A[], j)`. It returns the total number of cities that rained on the j-th day.

(b) (**10 marks**) Write a function `int predict(int A[], i, j)` that returns the prediction of rainfall of city i on the j-th day ($j \geq 7$).

The prediction rule is: city i is predicted to have rain (return `1`) if it rained on at least 3 of the previous 7 days ($j-1, j-2, \ldots, j-7$). Otherwise return `0`.

(c) (**10 marks**) Write a function `int sunniest_day(int A[])` that returns the sunniest day of the year. The sunniest day is defined as the day on which the smallest number of cities experienced rain. If there are multiple "sunniest days", printing any one of them is OK. NOTE: You can use functions in previous questions even if you did not answer them.

B2. (**30 marks**) A number d is said to be a *divisor* of a if d divides a exactly. A *perfect number* is a positive integer equal to the sum of its *proper divisors* (its positive divisors excluding itself). For example, 6 is perfect because its proper divisors are 1, 2, and 3, and $1 + 2 + 3 = 6$. A number a is *abundant* if the sum of its proper divisors is greater than a, and *deficient* if it is smaller.

In this question will write R functions to work with these definitions. You may find useful that the R's modulo operator works as follows:

```
> 2 %% 2
[1] 0
> 6 %% 3
[1] 0
> 3 %% 2
[1] 1
> 3 %% 4
[1] 3
> 9 %% 2:9
[1] 1 0 1 4 3 2 1 0
```

(a) (**8 marks**) Write an R function called `is_pd(a, d)` that takes two positive integers `a` and `d` and returns `TRUE` if `d` is a proper divisor of `a`, and `FALSE` otherwise.

(b) (**8 marks**) Using your function from question (a), write a function `sum_pds(a)` that returns the sum of all proper divisors of the positive integer `a`. Your implementation should include either a `for` or a `while` loop.

(c) (**5 marks**) Write a function `classify_number(a)` that returns the string `"perfect"`, `"abundant"`, or `"deficient"` according to the value of `a` and the result of `sum_pds(a)`. Briefly explain how your function handles the case `a = 1`.

(d) (**5 marks**) Define a function `classify_numbers_v(v)` that takes a vector of positive integers `v` and returns a character vector with the corresponding classification for each element of `v`. Your solution for this part should avoid explicit `for`/`while` loops, but you may use `sapply`, `lapply`, etc.

(e) (**4 marks**) Write a vectorised version of the function in question (b), called `sum_pds_v(a)`, that computes the sum of proper divisors of a scalar integer `a` without using any explicit `for`/`while` loops or calls to `sapply`, `lapply`, etc.